PROTOCOLO OFICIAL DE METODOLOGÍA ÁGIL (XP)

Guía de Desarrollo en Parejas

Pair Programming, Ciclo Ping-Pong TDD y Flujo Git en GitHub

INTEGRANTES DE LA PAREJA	METODOLOGÍA DE TRABAJO	REPOSITORIO OFICIAL
Cristian Beltrán & Desarrollador (Tú)	Extreme Programming (XP) · TDD	cristianbg2007/BackEnd_Medicent_v3

1. El Trabajo en Pareja para un Niño de 5 Años

🚀 Los Dos Pilotos de la Nave Espacial

Trabajar en pareja no significa que uno duerme mientras el otro trabaja. ¡Es como ser astronautas en una nave súper rápida!

- **El Conductor (Driver):** Es el piloto que tiene las dos manos en el volante y el pie en el acelerador. Él escribe en el teclado y hace que la nave avance rápido y derecho.
- **El Navegador (Navigator):** Es el copiloto que tiene el telescopio y los mapas estelares. Él no toca el volante, pero avisa: "*¡Cuidado con ese asteroide a la izquierda!*", "*¡Mira que aquí nos falta un punto y coma!*" y "*¡Buen trabajo, ahora tomemos la curva hacia el siguiente test!*".
- **La Campanita Mágica (Técnica Pomodoro):** Para que ninguno se canse, cada 25 o 30 minutos suena un relojito *¡Ring!* y cambian de asiento: el que manejaba ahora mira el mapa, y el copiloto toma el volante.

El Juego del Semáforo Mágico (TDD):

- 🔴 **Luz Roja (Compañero A):** Te hago una pregunta de adivinanza tan difícil que la computadora dice: "*¡Error! ¡No sé la respuesta!*". ¡Eso está perfecto porque la función aún no existe!
- 🟢 **Luz Verde (Compañero B):** Construyes la respuesta justa y necesaria con plastilina para que la prueba se ponga feliz y sonría en verde (OK).
- 🔵 **Luz Azul (Ambos):** Limpiamos la mesa, ordenamos los lápices de colores y dejamos el código brillante sin cambiar la respuesta.

El Cuaderno de Dibujo Compartido (Git y GitHub): GitHub es una biblioteca en la nube donde guardamos el libro de Medicent. Cada uno dibuja en su propia hoja de papel transparente (su rama `feature`) y cuando el dibujo está terminado y probado, el otro lo revisa con una lupa. Si está aprobado, se pega en el gran libro oficial.

2. Dinámica Técnica de Pair Programming (XP) 👨‍💻

> ⚡ **Reglas de Interacción entre Cristian y Tu Persona**
>
> Para maximizar la transferencia de conocimiento, evitar silos de información y garantizar cero deuda técnica:
>
> 1. **Enfoque Táctico vs. Estratégico:** El *Driver* piensa en la línea de código actual, en la sintaxis de Python y en cumplir la regla PEP 8. El *Navigator* piensa en la arquitectura por capas, en casos de borde (ej. división por cero, campos nulos, tokens expirados) y en la siguiente prueba unitaria.
> 2. **Comunicación Continua y en Voz Alta:** El Driver verbaliza lo que está escribiendo ("*Estoy declarando el método `calcular_tomas` con parámetros `fecha_inicio` y `frecuencia`...*"). El Navigator valida y retroalimenta al instante.
> 3. **Rotación Estricta:** Usar temporizador de 25-30 minutos (bloques Pomodoro) con 5 minutos de descanso activo.

3. Metodología TDD: Ciclo Ping-Pong Riguroso 🧪

Fase	Responsable	Acción Técnica en el Código	Resultado Esperado
1. ROJA 🔴	Compañero A (Driver)	Escribe un método de prueba en `tests/test_servicios.py` que verifique un requerimiento no implementado aún.	El test debe fallar con `AssertionError` o `AttributeError`.
2. VERDE 🟢	Compañero B (Driver)	Escribe en `app/services/` el código mínimo estricto para que la aserción sea verdadera.	El comando de pruebas retorna `Ran X tests — OK` al 100%.
3. REFACTOR 🔵	Ambos en conjunto	Revisan nomenclatura en español, eliminan duplicidad, optimizan queries y validan estándares PEP 8.	Todos los tests continúan ejecutándose en verde con cero regresiones.

Aislamiento de Pruebas: SQLite en Memoria (`tests/base_test.py`)

Para que la suite de pruebas se ejecute en milisegundos y no dependa del servidor MySQL local, se utiliza una base de datos en memoria:

```
# tests/base_test.py
import unittest
from app import create_app
from app.database.connection import db

class BaseTestCase(unittest.TestCase):
    def setUp(self):
        self.app = create_app("app.config.settings.TestConfig")
        self.client = self.app.test_client()
        self.ctx = self.app.app_context()
        self.ctx.push()
        db.create_all()

    def tearDown(self):
        db.session.remove()
        db.drop_all()
        self.ctx.pop()
```

Comando Oficial de Ejecución de Pruebas:

```
py -m unittest discover -s tests -p "test_*.py"
```

4. Matriz de Distribución de Responsabilidades 📋

Integrante	Módulo Responsable	Archivos a Implementar	Alcance Funcional Clave
Cristian Beltrán	**Seguridad, Autenticación y Usuarios (IAM)**	`models/usuario.py` `models/paciente_cuidador.py`	Modelo ORM `Usuario` con hashing de contraseñas y tabla asociativa N:M.

		`services/auth_service.py`
		`controllers/auth_controller.py`
		`routes/auth_routes.py`
			Registro, validación de credenciales, emisión de Token JWT con claims de rol y asignación de cuidadores a pacientes.

		`middlewares/auth_middleware.py` `tests/test_modelos.py` (auth)	Decorador `@rol_requerido` para proteger endpoints y pruebas TDD de autenticación y permisos.

Tu Persona (Compañero)	**Módulo Clínico, Tratamientos y Motor de Tomas**	`models/medicamento.py`
		`models/tratamiento.py`
		`models/registro_toma.py`
			Modelado de fármacos, prescripciones médicas y registros de dosis individuales.

		`services/tratamiento_service.py`
		`services/toma_service.py`
		`controllers/*` y `routes/*`
			Motor algorítmico de generación automática de tomas según intervalos horarios y tracking de estados de tomas (tomada, omitida, retrasada).

		`tests/test_servicios.py` (clínico)	Pruebas TDD del motor de calendarización horaria y transiciones de estados.

Tareas Conjuntas (100% en Pareja)	**Arquitectura Base y Pruebas End-to-End**	`database/schema.sql`
		`app/__init__.py`
		`tests/test_rutas_api.py`
			Diseño DDL inicial de MySQL, configuración de Application Factory con CORS y pruebas E2E de integración de todo el flujo.

5. Protocolo de Git y Enlace al Repositorio Remoto 🌐

> 🔗 **Repositorio Central de GitHub**
>
> URL Oficial: **https://github.com/cristianbg2007/BackEnd_Medicent_v3.git**

Comandos Paso a Paso para Enlazar el Repositorio Local

Ejecutar los siguientes comandos desde la raíz del proyecto local (`Backend_Medicent_v3`):

```
# 1. Inicializar repositorio Git local
git init

# 2. Configurar rama principal estándar
git branch -M main

# 3. Vincular el repositorio remoto oficial
git remote add origin https://github.com/cristianbg2007/BackEnd_Medicent_v3.git

# 4. Verificar la vinculación correcta
git remote -v
```

```
# 5. Crear la rama de desarrollo continuo
git checkout -b develop
```

Estrategia de Ramas de Trabajo

- `main`: Rama protegida de producción. Solo recibe merges estables aprobados.
- `develop`: Rama integradora de desarrollo del equipo.
- `feature/database-schema`: Rama conjunta para el diseño y scripts de base de datos.
- `feature/auth-usuarios`: Rama exclusiva para las tareas de Cristian.
- `feature/tratamientos-tomas`: Rama exclusiva para tus tareas clínicas.

Formato de Commits con Co-Autoría Obligatoria

```
git commit -m "feat(tratamientos): implementar motor automatico de tomas

- Calculo de intervalos segun frecuencia_horas
- Cobertura de pruebas unitarias al 100%

Co-authored-by: Cristian <cristian@medicent.com>
Co-authored-by: Desarrollador <tu_correo@medicent.com>"
```

Criterios de Aceptación (Definition of Done - DoD)

- ✓ 100% de tablas, campos, modelos, servicios y rutas rigurosamente en español.
- ✓ Cada función cuenta con su prueba unitaria desarrollada bajo TDD.
- ✓ La suite de pruebas completa se ejecuta exitosa y sin fallos (`OK`).
- ✓ Respuestas HTTP con códigos estándar (200, 201, 400, 401, 403, 404).
- ✓ Todo commit incluye la co-autoría de Cristian Beltrán y tu persona.
- ✓ El merge a `develop` requiere revisión y aprobación cruzada mediante Pull Request.